Database Compression Using Information Theory

Finlay Peden



MInf Project (Part 1) Report
Master of Informatics
School of Informatics
University of Edinburgh

2026

Abstract

This skeleton demonstrates how to use the `infthesis` style for undergraduate dissertations in the School of Informatics. It also emphasises the page limit, and that you must not deviate from the required style. The file `skeleton.tex` generates this document and should be used as a starting point for your thesis. Replace this abstract text with a concise summary of your report.

Research Ethics Approval

This project was planned in accordance with the Informatics Research Ethics policy. It did not involve any aspects that required approval from the Informatics Research Ethics committee.

Declaration

I declare that this thesis was composed by myself, that the work contained herein is my own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

(*Finlay Peden*)

Acknowledgements

Any acknowledgements go here.

Table of Contents

1 Introduction **1**
1.1 Using Sections 2
1.2 Citations 2

2 Background **3**
2.1 Running Example 3
2.2 Acyclic Schemas and Multivalued Dependencies 4
2.3 Information Theory 5
2.4 Mining approximate acyclic schemes from relation 6
2.5 Database Compression Techniques 7

3 Conclusions **9**
3.1 Final Reminder 9

Bibliography **10**

A First appendix **12**
A.1 First section 12

B Participants' information sheet **13**

C Participants' consent form **14**

Chapter 1

Introduction

The preliminary material of your report should contain:

- The title page.
- An abstract page.
- Declaration of ethics and own work.
- Optionally an acknowledgements page.
- The table of contents.

As in this example `skeleton.tex`, the above material should be included between:

```
\begin{preliminary}
    ...
\end{preliminary}
```

This style file uses roman numeral page numbers for the preliminary material.

The main content of the dissertation, starting with the first chapter, starts with page 1. ***The main content must not go beyond page 40.***

The report then contains a bibliography and any appendices, which may go beyond page 40. The appendices are only for any supporting material that's important to go on record. However, you cannot assume markers of dissertations will read them.

You may not change the dissertation format (e.g., reduce the font size, change the margins, or reduce the line spacing from the default single spacing). Be careful if you copy-paste packages into your document preamble from elsewhere. Some LaTeX packages, such as `fullpage` or `savetrees`, change the margins of your document. Do not include them!

Over-length or incorrectly-formatted dissertations will not be accepted and you would have to modify your dissertation and resubmit. You cannot assume we will check your submission before the final deadline and if it requires resubmission after the deadline to conform to the page and style requirements you will be subject to the usual late penalties based on your final submission time.

1.1 Using Sections

Divide your chapters into sub-parts as appropriate.

1.2 Citations

When citing work using author names, your sentences should still read as correct English if any parts in parenthesis are removed. Use `\citet` when citing the name as text that's part of your sentence:

> We follow the method of Arimura [1].

and use `\citep` for a parenthetical citation:

> It's possible to learn first-order Horn sequences from entailment [4].

You may use any consistent reference style that you prefer, including "[1]" numerical citations.

Chapter 2

Background

In this chapter, we will give a general knowledge of the methods needed to understand our proposed compression approach and how to measure its effectiveness. We will start by discussing acyclic schemas along with Multivalued Dependencies (MVDs) and how they will be used to compress the data. We will then explain how the definition of entropy from information theory can be used in determining data loss during the compression. Then we summarize the work by Kenig et al. [7] in which they describe systems to discover MVDs and Acyclic Join Dependencies. Finally, we will briefly describe widely used database compression techniques.

To keep consistency we will use the same notation as Kenig et al. [7]. Let Ω be a set of variables, also called attributes. If $X, Y \subseteq \Omega$, then XY denotes $X \cup Y$.

2.1 Running Example

We will use (Figure 2.1) as an example to demonstrate the information about the background topics. We consider the relation R over the schema $\Omega = \{A, B, C, D\}$. As illustrated, the table R can be decomposed into smaller tables, and joining these tables reconstructs R. The join dependency is $R = R[AB] \bowtie R[AC] \bowtie R[BD]$. This dependency only holds for the rows in black. The red rows will be used as an example of approximate acyclic schemas in Section 2.4.

R:

A	B	C	D	
a_1	b_1	c_1	d_1	1/4
a_1	b_2	c_1	d_2	1/4
a_2	b_1	c_2	d_1	1/4
a_2	b_2	c_2	d_2	1/4
a_1	b_2	c_2	d_2	

$=$

A	B	
a_1	b_1	1/4
a_1	b_2	1/4
a_2	b_1	1/4
a_2	b_2	1/4

$\bowtie$

A	C	
a_1	c_1	1/2
a_2	c_2	1/2
a_1	c_2	

$\bowtie$

B	D	
b_1	d_1	1/2
b_2	d_2	1/2

Figure 2.1: A relation R and its decomposition into an acyclic schema

2.2 Acyclic Schemas and Multivalued Dependencies

In this section, ignore the probabilities (discussed in Sec. 2.3) and the red rows (discussed in Sec. 2.4) in Figure 2.1.

For $Y \subseteq \Omega$ we let $R[Y]$ denote the projection of R onto the attributes Y.

When trying to compress databases, it is often useful to decompose a table R with attributes Ω into smaller tables. A schema is a set $S = \{\Omega_1, \ldots, \Omega_k\}$ of attribute sets whose union is Ω, where no set is contained in another, since a contained set adds no information to the join. R satisfies the join dependency $\mathrm{JD}(S)$, written $R \models \mathrm{JD}(S)$, when joining the projections $R[\Omega_1], \ldots, R[\Omega_k]$ gives us exactly the table R. This also means that the decomposition is lossless as joining the projections gives exactly R.

An MVD is a special type of join dependency in which, for each value of a set of attributes X, the remaining attributes split into groups that vary independently of one another. Let $X, Y_1, \ldots, Y_m \subseteq \Omega$. We say R satisfies the multivalued dependency $X \twoheadrightarrow Y_1|Y_2|\ldots|Y_m$ if $R = R[XY_1] \bowtie \cdots \bowtie R[XY_m]$. Where $m \geq 2$, the Y_is are pairwise disjoint and $XY_1 \cdots Y_m = \Omega$. We call X the *key* and $\{Y_1, \ldots, Y_m\}$ the *dependents*. When $m = 2$ we call it a *standard MVD* which is the most common form in the literature.

A schema is *acyclic* if it has a join tree. The schema $\{AB, AC, BD\}$ of Figure 2.1 is acyclic, as shown by the join tree in Figure 2.2.

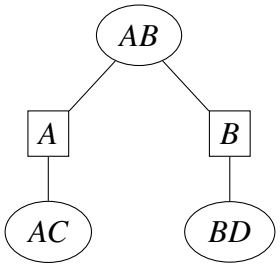


Figure 2.2: Join Tree

An acyclic schema can be fully described by a set of MVDs, one for each edge of the join tree. The acyclic schema can be described by these two multivalued dependencies: $B \twoheadrightarrow D|AC$, $A \twoheadrightarrow C|BD$. For example the edge $AB \overset{\mathrm{B}}{-} BD$ in the join tree defines the MVD $B \twoheadrightarrow D|AC$. The *separator* of an edge is the intersection of the two bags it joins. Removing an edge splits the tree in two, and the attributes outside the separator split accordingly. The separator and these two groups give an MVD whose key is the separator.

The *separator* of an edge of a join tree is the intersection of the two bags it joins. Removing the edge splits the tree into two subtrees, and the attributes of each subtree that are not in the separator form the two dependents of an MVD whose key is the separator. In Figure 2.2, the edge between AB and BD has separator B and gives the MVD $B \twoheadrightarrow D|AC$, and the edge between AB and AC has separator A and gives the MVD $A \twoheadrightarrow C|BD$. Beeri et al. [3] showed that R satisfies the acyclic join dependency of the schema exactly when it satisfies the MVD of every edge.

We will describe a *join tree* using the definition from Kenig et al. [7] as we will use the same notation for join trees when computing entropy.

Definition 2.2.1 *A* join tree *is a pair* $(\mathcal{T},\chi)$ *where* $\mathcal{T}$ *is an undirected tree, and* χ *is a function that maps each* $u \in \text{nodes}(\mathcal{T})$ *to a set of variables* $\chi(u)$*, called a* bag*, such that the following* running intersection *property holds: for every variable* X*, the set* $\{u \in \text{nodes}(\mathcal{T}) \mid X \in \chi(u)\}$ *is a connected component of* $\mathcal{T}$*. We denote by* $\chi(\mathcal{T}) \stackrel{\text{def}}{=} \bigcup_u \chi(u)$*, the set of variables of the join tree.*

We will often denote the join tree as $\mathcal{T}$ when $\mathcal{X}$ is clear from the context. The bags of $\mathcal{T}$ are the attributes $\Omega_1, \ldots \Omega_k$, so $\mathcal{T}$ determines the schema $S = \{\Omega_1, \ldots \Omega_m\}$.

We say that R satisfies the *acyclic join dependency* $\text{AJD}(S)$, written $R \models \text{AJD}(S)$, if the schema S is acyclic and $R \models \text{JD}(S)$. A single MVD $X \twoheadrightarrow Y_1|\cdots|Y_m$ is the simplest case, since it is the join dependency of the schema $\{XY_1, \ldots, XY_m\}$, whose sets all intersect in X. This also explains why one MVD with several dependents is a compact way of writing several standard ones: for example, $X \twoheadrightarrow A|B|C$ holds exactly when $X \twoheadrightarrow AB|C$, $X \twoheadrightarrow A|BC$ and $X \twoheadrightarrow AC|B$ all hold [2].

The set of MVDs obtained from the edges of a join tree $\mathcal{T}$ is called the *support* of $\mathcal{T}$. Since a tree with k nodes has $k-1$ edges, the support contains $k-1$ MVDs. Because no set of S may be contained in another, an acyclic schema over n attributes has at most n sets [3], so a decomposition of R into an acyclic schema never produces more tables than R has attributes. In Figure 2.2 the support consists of two MVDs and the schema has three sets, which is less than the 4 attributes of R.

2.3 Information Theory

Information theory is the mathematics of quantifying and measuring information. In our context we will be using it to measure the uncertainty in data. Entropy is the main component of information theory that is relevant to this project. It is used to calculate how uncertain a random variable is.

Let X be a discrete random variable with a set of possible values $\mathcal{X}$ and probability mass function $p(x)$ such that the sum of all probabilities is equal to one. The entropy of X is defined as:

$$H(X) = \sum_{x \in \mathcal{X}} p(x) \log \frac{1}{p(x)} \tag{2.1}$$

Mutual Information is the measure of mutual dependence between two random variables. Let $A, B, C \subseteq \Omega$. We write it as:

$$I(B;C|A) = H(AB) + H(AC) - H(ABC) - H(A) \tag{2.2}$$

$I(B;C|A) = 0$ iff the variables B and C are conditionally independent given A.

Kenig et al. [7] uses the entropy of the *empirical distribution* which is the uniform distribution of it tuples in which all data in the relation R has an equal probability of

$\frac{1}{N}$. Where R is a relationship with attributes $\Omega = \{X_1, \ldots, X_n\}$ and N is the number of tuples. For any subset of variables $\alpha \subseteq \{1, \ldots, n\}$, we denote by X_α the set of variables X_i where $i \in \alpha$, and by $R(X_\alpha = x_\alpha)$ the subset of tuples $t \in R$ where $t[X_\alpha] = x_\alpha$ for fixed values x_α. Due to the uniformity of the empirical distribution, the marginal probability of this subset is expressed as:

$$p(X_\alpha = x_\alpha) = \frac{|R(X_\alpha = x_\alpha)|}{N} \tag{2.3}$$

Therefore we can state the entropy:

$$H(X_\alpha) \stackrel{\text{def}}{=} \log N - \frac{1}{N} \sum_{x_\alpha \in \mathcal{D}_\alpha} |R(X_\alpha = x_\alpha)| \log |R(X_\alpha = x_\alpha)| \tag{2.4}$$

where $\mathcal{D}_\alpha$ represents the domain of possible values for X_α.

Which Kenig et al. [7] notes can be computed by the SQL query:

$$\texttt{Select } X_\alpha, \texttt{count(*)} \times \log(\texttt{count(*)}) \texttt{ From } R \texttt{ Group By } X_\alpha.$$

Following the work done by Lee [8, 9] as presented by Kenig et al. [7]. We define the following expression to be the connection between entropic measures and database constraints:

$$\mathcal{J}(\mathcal{T}, \chi) \stackrel{\text{def}}{=} \sum_{v \in \text{nodes}(\mathcal{T})} H(\chi(v)) - \sum_{(v_1, v_2) \in \text{edges}(\mathcal{T})} H(\chi(v_1) \cap \chi(v_2)) - H(\chi(\mathcal{T})) \tag{2.6}$$

We abbreviate it with $\mathcal{J}(\mathcal{T})$, or $\mathcal{J}$, when $\mathcal{T}, \chi$ are clear from the context. Lee proved that $\mathcal{J}$ does not depend on the tree itself but instead relies on the schema S defined by the tree. This allows us to denote $\mathcal{J}(\mathcal{T})$ as $\mathcal{J}(S)$ if S is an acyclic schema.

We will also state the theorem in Kenig et al. [7] which provides us with a relationship between MVDs and mutual information. The theorem is:

Theorem 2.3.1 ([9]) *Let H be the entropy of the empirical distribution on R, and let* $\mathbf{S}$ *be any acyclic schema. Then $R \models \text{AJD}(\mathbf{S})$ iff $\mathcal{J}(\mathbf{S}) = 0$.*

For a standard MVD this theorem implies that $R \models X \twoheadrightarrow Y|Z$ iff $I(Y;Z|X) = 0$.

2.4 Mining approximate acyclic schemes from relation

Maimon [7](Multivalued Approximate Inference Mining and NOrmalization) is the system proposed by Kenig et al. [7] to discover MVDs and acyclic schemes in data. When evaluating their work they found that this method could be scaled to 1 million rows and 30 attributes with small amounts of spurious tuples. Where a spurious tuple is a tuple not in the original data. Their use of approximate MVDs and approximate

acyclic schemas allowed them to consider a schema valid even though it may not be a correct decomposition of a relation R. This allows us to use these approximate schemas for many applications such as database compression.

If we now look at the red rows in the running example we can see that if we add the 5th red row to R then we also need to add the row a_1, c_1 in $R[AC]$ as well. However, this leads to a spurious tuple after we compute the join $R[AB] \bowtie R[AC] \bowtie R[BD]$. The spurious tuple that is not in R is (a_1, b_1, c_2, d_1). In the paper they observe this and note that if only perfect MVDs and acyclic schema were taken then it would not apply widely to databases as if one row in the database changed then the whole schema may no longer be valid. Therefore they chose to use approximate MVDs and acyclic schema instead. They use $\mathcal{J}$ to compute whether a schema is approximate. They call these *approximate schema* ε *-schema* if $\mathcal{J}(S) \leq \varepsilon$. The notation they use is, $R \models_\varepsilon \text{AJD}(S)$.

They used three main techniques to efficiently create schema and design their enumeration algorithms. The first technique is to be able to determine approximate acyclic schema from approximate MVDs. The second technique they describe is a way to reduce the number of MVDs by only discovering full MVDs, and finally they only discover full MVDs that have a minimal separator as a key.

In the first technique they use an identity and a shannon inequality to discover approximate schemas and MVDs. With the second technique they create a notation for one MVD refining another MVD. If we have two MVDs $\phi = X \twoheadrightarrow A_1 | \ldots | A_m$ and $\psi = X \twoheadrightarrow B_1 | \ldots | B_k$. We say ϕ *refines* ψ if they both have the same key and for every A_i that is an element of the *dependents* of ϕ there exists a B_j that is an element of the dependents of ψ such that $A_i \subseteq B_j$. We denote this relationship as $\phi \succeq \psi$. They denote $\text{FULLMVD}_\varepsilon(R, X)$ to be the set of all full $\varepsilon -$ MVDs with the key X. They describe an MVD ψ as full, if $R \models_\varepsilon \psi$, and for all strict refinements $\phi \succ \psi$ $R \not\models_\varepsilon \phi$. Finally they define *minimal separators* which are small attribute sets that make two attributes approximately independent. See Definition 5.5 in Kenig et al. [7] for the formal definition.

Then the paper describes the two main phases of their **Maimon** system and the algorithms corresponding to these phases. The first phase uses the algorithm **MVDMiner** which discovers $\varepsilon-$MVDs over a relation R. This is also the section where they compute the entropy $H(X)$ with their procedure $getEntropy_R(X)$. Which they have noted is the most computationally expensive part of the **Maimon** system.

The second phase of the **Maimon** system computes acyclic $\varepsilon-$ schemes from the set of full $\varepsilon-$ MVDs discovered in the first phase. The algorithm they use in this phase is **ASMiner** which will generate an acyclic schema for all the $\varepsilon-$ MVDs returned from the **MVDMiner**.

2.5 Database Compression Techniques

In this section we will give a brief overview of some general database compression techniques that give a better understanding of the topic of database compression and give an option to compress our data further when it can no longer be decomposed using

the techniques in [7].

When trying to compress data we can either have lossless or lossy compression. *Lossless* compression is when the data is compressed and decompression returns exactly the original data. This is useful in databases as there are large amounts of data that need to be compressed while retaining all the information in the database. *Lossy* compression is used when a small amount of data loss is acceptable such as in JPEG image files. Lossy compression is used as it allows for much smaller sizes of data once compressed.

I will define some basic metrics to measure the performance of compression techniques from [10]. The compression ratio is:

$$\text{CR} = \frac{\text{data size without compression}}{\text{data size with compression}} \tag{2.7}$$

where the unit of data depends on the application it is used in. From this compression ratio we can also work out the expected reduction in data size with this equation:

$$\text{REDUCTION} = (1 - \frac{1}{\text{CR}}) \times 100\% \tag{2.8}$$

Dictionary based compression methods are widely used for lossless database compression. The general idea behind dictionary based methods is to identify repeated values or sequences of data and replace them with a code that is shorter than the original data. Decompressing the data replaces the codes for the original data. Some examples of dictionary based methods are the LZ4 compression algorithm [5] and the LZW compression algorithm [11].

Huffman coding [6] is another compression method that is commonly used alongside other techniques for lossless compression. The general idea is that variable length codes are assigned to the data based on how frequently the data appears, with more frequent values getting shorter codes. The coding ensures that all codes have a different prefix which allows the data to be decoded without any ambiguity.

Other popular techniques for database compression include null suppression, which omits NULL values or runs of zeros and blanks and records where they were, and run-length encoding, which stores a value together with the number of times it repeats.

Chapter 3

Conclusions

3.1 Final Reminder

The body of your dissertation, before the references and any appendices, *must* finish by page 40. The introduction, after preliminary material, should have started on page 1.

You may not change the dissertation format (e.g., reduce the font size, change the margins, or reduce the line spacing from the default single spacing). Be careful if you copy-paste packages into your document preamble from elsewhere. Some LaTeX packages, such as `fullpage` or `savetrees`, change the margins of your document. Do not include them!

Over-length or incorrectly-formatted dissertations will not be accepted and you would have to modify your dissertation and resubmit. You cannot assume we will check your submission before the final deadline and if it requires resubmission after the deadline to conform to the page and style requirements you will be subject to the usual late penalties based on your final submission time.

Bibliography

[1] Hiroki Arimura. Learning acyclic first-order horn sentences from entailment. In *Proc. of the 8th Intl. Conf. on Algorithmic Learning Theory, ALT '97*, pages 432–445, 1997.

[2] Catriel Beeri, Ronald Fagin, and John H. Howard. A complete axiomatization for functional and multivalued dependencies in database relations. In Diane C. P. Smith, editor, *Proceedings of the 1977 ACM SIGMOD International Conference on Management of Data, Toronto, Canada, August 3-5, 1977*, pages 47–61. ACM, 1977. doi: 10.1145/509404.509414. URL https://doi.org/10.1145/509404.509414.

[3] Catriel Beeri, Ronald Fagin, David Maier, and Mihalis Yannakakis. On the desirability of acyclic database schemes. *J. ACM*, 30(3):479–513, 1983. doi: 10.1145/2402.322389. URL https://doi.org/10.1145/2402.322389.

[4] Chen-Chung Chang and H. Jerome Keisler. *Model Theory*. North-Holland, third edition, 1990.

[5] Yann Collet. LZ4: Extremely fast compression algorithm. https://github.com/lz4/lz4. Accessed: ¡date you checked it¿.

[6] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952. doi: 10.1109/JRPROC.1952.273898.

[7] Batya Kenig, Pranay Mundra, Guna Prasaad, Babak Salimi, and Dan Suciu. Mining approximate acyclic schemes from relations. In David Maier, Rachel Pottinger, AnHai Doan, Wang-Chiew Tan, Abdussalam Alawini, and Hung Q. Ngo, editors, *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020*, pages 297–312. ACM, 2020. doi: 10.1145/3318464.3380573. URL https://doi.org/10.1145/3318464.3380573.

[8] Tony T. Lee. An information-theoretic analysis of relational databases - part I: data dependencies and information metric. *IEEE Trans. Software Eng.*, 13(10):1049–1061, 1987. doi: 10.1109/TSE.1987.232847. URL https://doi.org/10.1109/TSE.1987.232847.

[9] Tony T. Lee. An information-theoretic analysis of relational databases - part II: information structures of database schemas. *IEEE Trans. Software*

Eng., 13(10):1061–1072, 1987. doi: 10.1109/TSE.1987.232848. URL https://doi.org/10.1109/TSE.1987.232848.

[10] Mark A. Roth and Scott J. Van Horn. Database compression. *SIGMOD Rec.*, 22(3):31–39, 1993. doi: 10.1145/163090.163096. URL https://doi.org/10.1145/163090.163096.

[11] Terry A. Welch. A technique for high-performance data compression. *Computer*, 17(6):8–19, 1984. doi: 10.1109/MC.1984.1659158. URL https://doi.org/10.1109/MC.1984.1659158.

Appendix A

First appendix

A.1 First section

Any appendices, including any required ethics information, should be included after the references.

Markers do not have to consider appendices. Make sure that your contributions are made clear in the main body of the dissertation (within the page limit).

Appendix B

Participants' information sheet

If you had human participants, include key information that they were given in an appendix, and point to it from the ethics declaration.

Appendix C

Participants' consent form

If you had human participants, include information about how consent was gathered in an appendix, and point to it from the ethics declaration. This information is often a copy of a consent form.